

ROS2 - CLI newbie

Intelligent Robotics I

Andrés Suárez García

CC BY-SA 4.0

University of Vigo

Environment	2
RQT & Turtlesim	5
Nodes - understanding and running nodes	14
Topics	21
Services	36
Actions	47
Glossary of Terms	59

Environment



- ROS2 uses workspaces to organize development
- main workspace is “underlay”
- local ones are “overlays”

- sourcing setup files is always required

```
source /opt/ros/humble/setup.bash
```

- add sourcing to shell startup script to avoid repeat

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
```

- check environment variables

```
printenv | grep -i ROS
```

```
# -----
```

```
ROS_VERSION=2
```

```
ROS_PYTHON_VERSION=3
```

```
ROS_DISTRO=humble
```

RQT & Turtlesim

- `ros2` tool manages and interacts with ROS 2 systems using commands.
- `rqt` is a GUI tool that makes interacting with ROS 2 easier than the command line.
- `turtlesim` is a simple simulator for learning ROS 2 basics

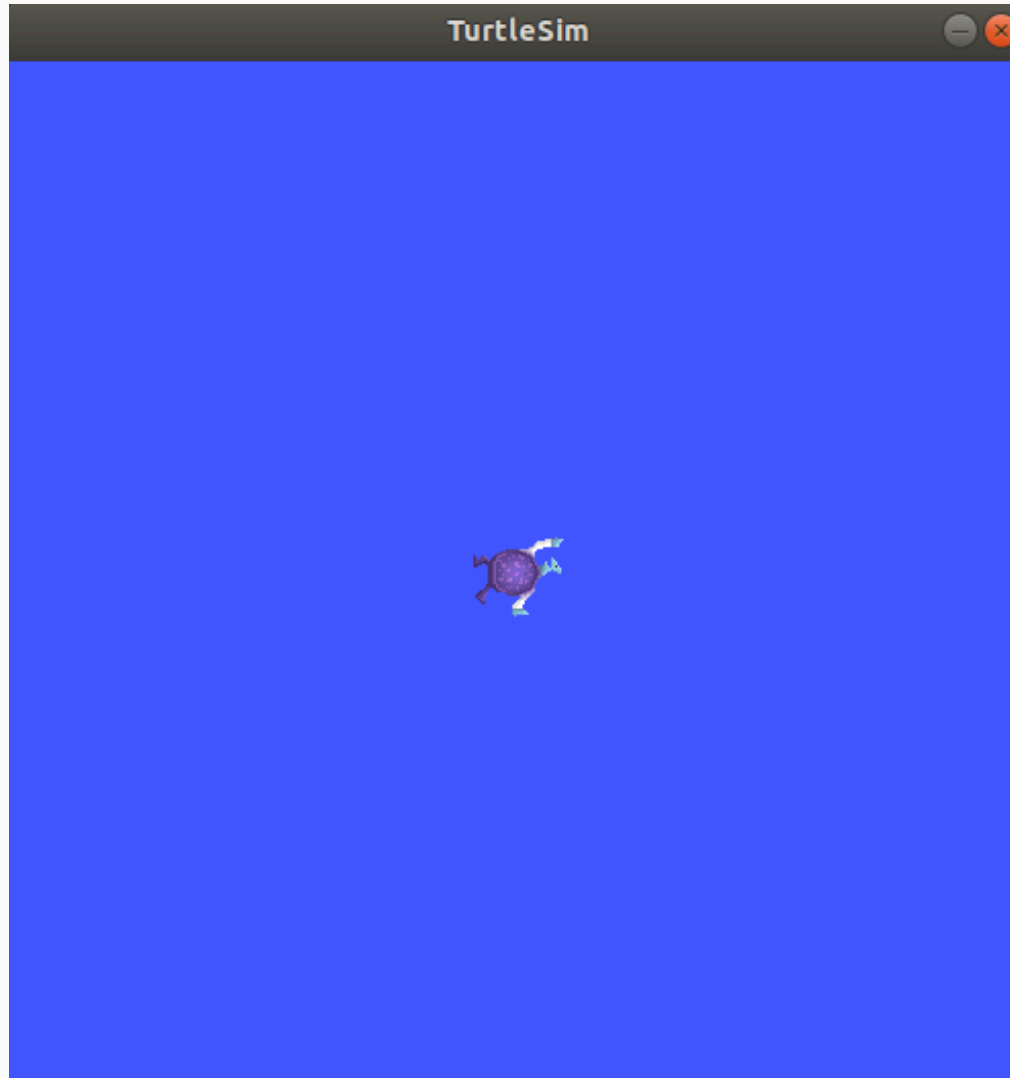
Install and check.

```
# ----- terminal 1 ----- #  
sudo apt install ros-humble-turtlesim  
ros2 pkg executables turtlesim  
# .....  
turtlesim draw_square  
turtlesim mimic  
turtlesim turtle_teleop_key  
turtlesim turtlesim_node
```


start turtlesim

Start a node with turtle sim

```
# ----- terminal 1 ----- #  
ros2 run turtlesim turtlesim_node
```



Run a new node to control the first node.

```
# ----- terminal 2 ----- #  
ros2 run turtlesim turtle_teleop_key
```

list shows the active components:

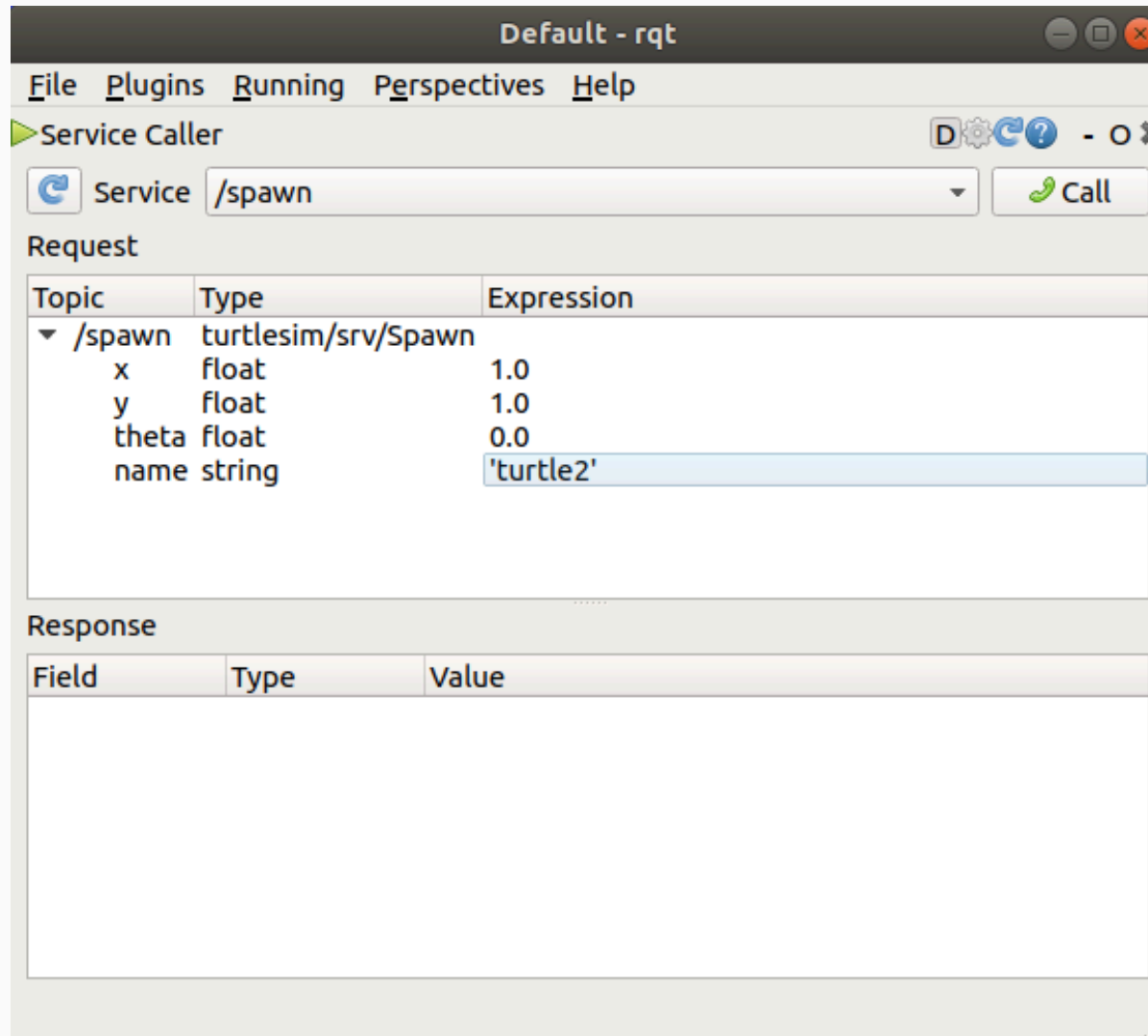
```
# ----- terminal 3 ----- #  
ros2 node list  
ros2 topic list  
ros2 service list  
ros2 action list
```

Install rqt and its plugins.

```
sudo apt update  
sudo apt install 'ros-humble-rqt*'  
rqt
```

use rqt

- select “Plugins > Services > Service Caller”
- write ‘turtle2’ in the ‘name’ field
- spawn a turtle selecting Service “/spawn” and click “Call”



- 'turtle 2' needs a second teleop node to control it
- remapping it is needed
- only the terminal active will receive key inputs

```
# ----- terminal 4 ----- #  
ros2 run turtlesim turtle_teleop_key \  
--ros-args --remap turtle1/cmd_vel:=turtle2/cmd_vel
```

To stop the simulation:

- Ctrl + c in turtlesim_node terminal
- q in turtle_teleop_key terminal

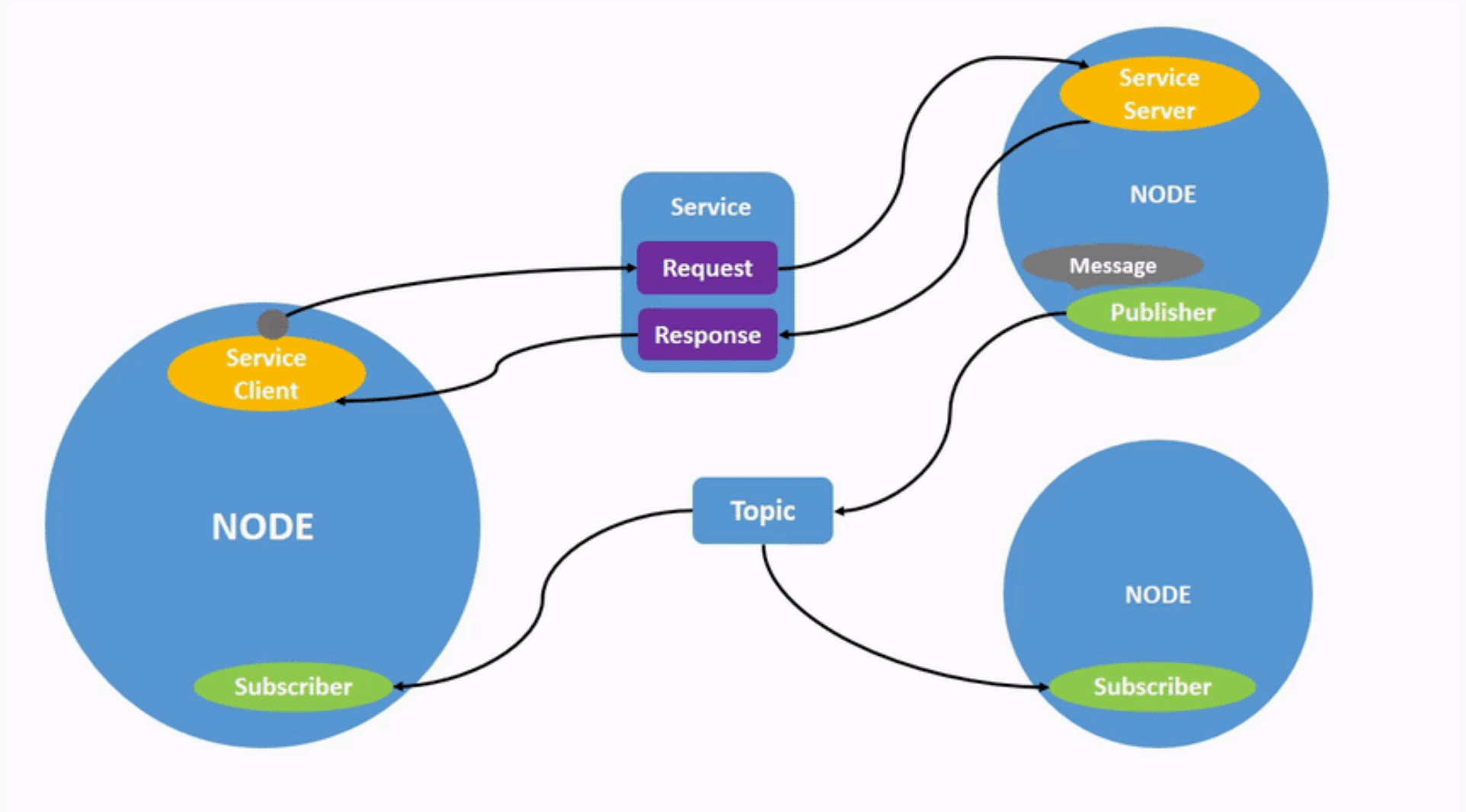
Nodes - understanding and running nodes

What?

- a robotic system is made of modular processes
- each node should be responsible for a single process
- each node can send and receive data from other nodes

via: topics, services or actions

communication between nodes



ros2 run launches executable from package

```
# ----- terminal 1 ----- #  
# ros2 run <package_name> <executable_name>  
ros2 run turtlesim turtlesim_node
```

```
# ----- terminal 2 ----- #  
ros2 run turtlesim turtle_teleop_key
```

ros2 node list show names all active nodes

```
# ----- terminal 3 ----- #
```

```
ros2 node list
```

```
# -----
```

```
/turtlesim
```

```
/teleop_turtle
```

Remapping allows to reassign node properties (node name, topic names, etc.)

```
# ----- terminal 3 ----- #  
# reassign name /turtlesim node  
ros2 run turtlesim turtlesim_node \  
--ros-args --remap __node:=my_turtle
```

```
# ----- terminal 4 ----- #  
ros2 node list  
# -----  
/my_turtle  
/turtlesim  
/teleop_turtle
```

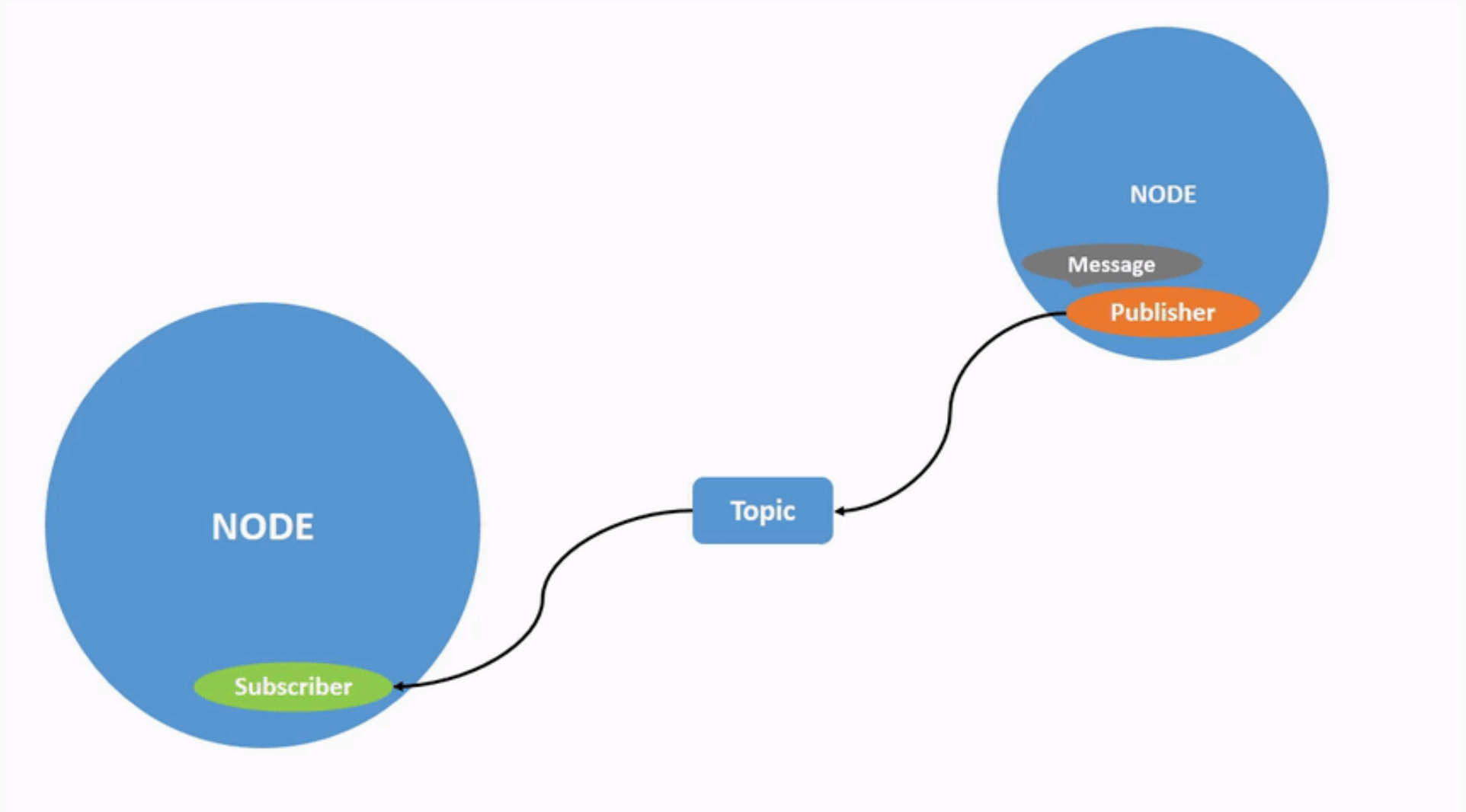
Returns a list of subscribes, publishers, services and actions

```
# ----- terminal 4 ----- #
# ros2 node info <node_name>
ros2 node info /my_turtle
# -----
/my_turtle
Subscribers:
  /turtle1/cmd_vel: geometry_msgs/msg/Twist
Publishers:
  /rosout: rcl_interfaces/msg/Log
  ...
Service Servers:
  /clear: std_srvs/srv/Empty
  ...
Service Clients:

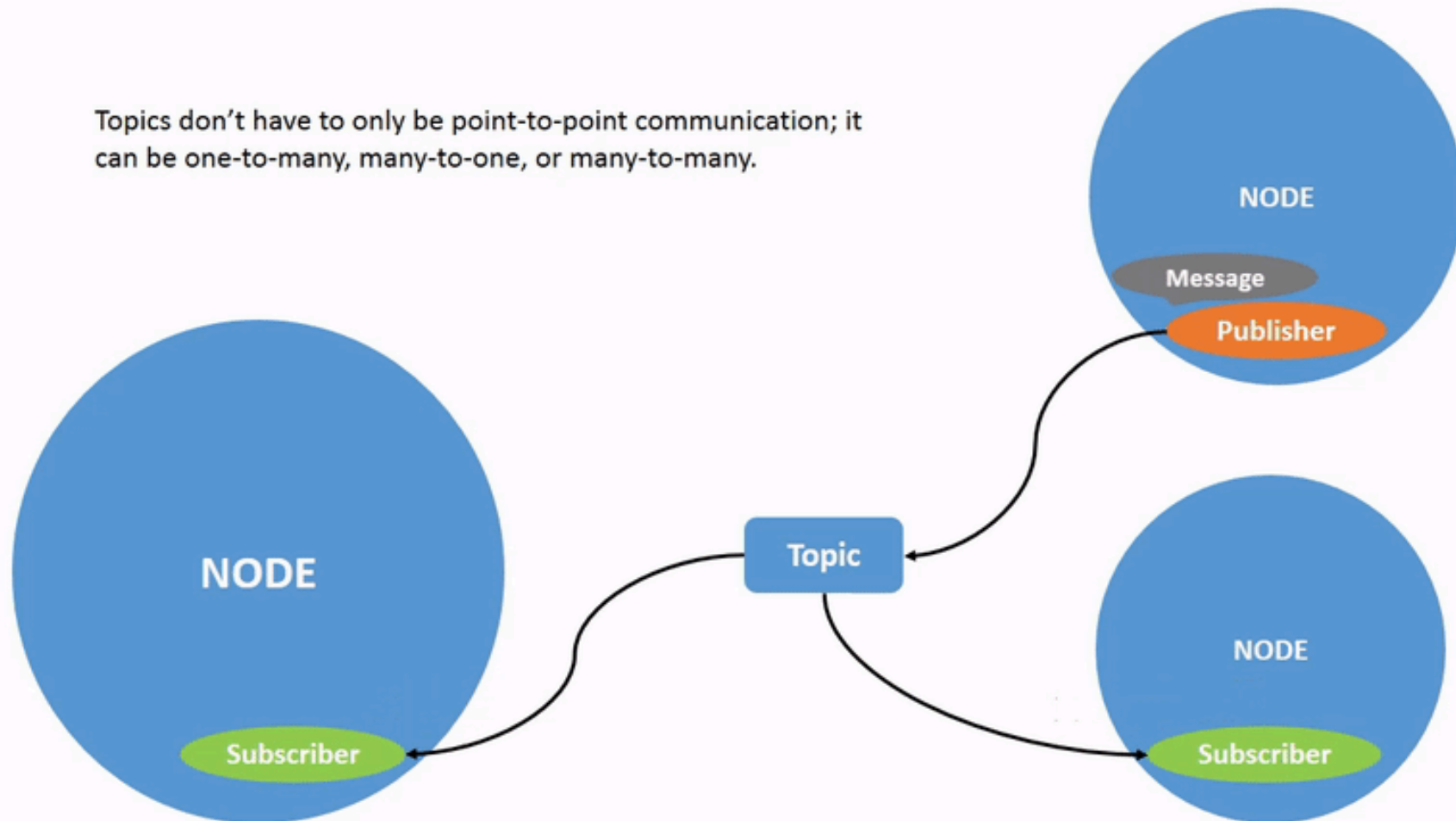
Action Servers:
  /turtle1/rotate_absolute: turtlesim/action/RotateAbsolute
  ...
```

Topics

- method of communication between nodes
- based on publisher-subscriber model
- continuous data streams
- asynchronous



Topics don't have to only be point-to-point communication; it can be one-to-many, many-to-one, or many-to-many.



```
# ----- terminal 1 ----- #  
ros2 run turtlesim turtlesim_node  
  
# ----- terminal 2 ----- #  
ros2 run turtlesim turtle_teleop_key  
  
# ----- terminal 3 ----- #  
ros2 node list  
# -----  
/turtlesim  
/teleop_turtle
```

Helps to visualize nodes and connections between (topics, services, actions).

```
# ----- terminal 3 ----- #
```

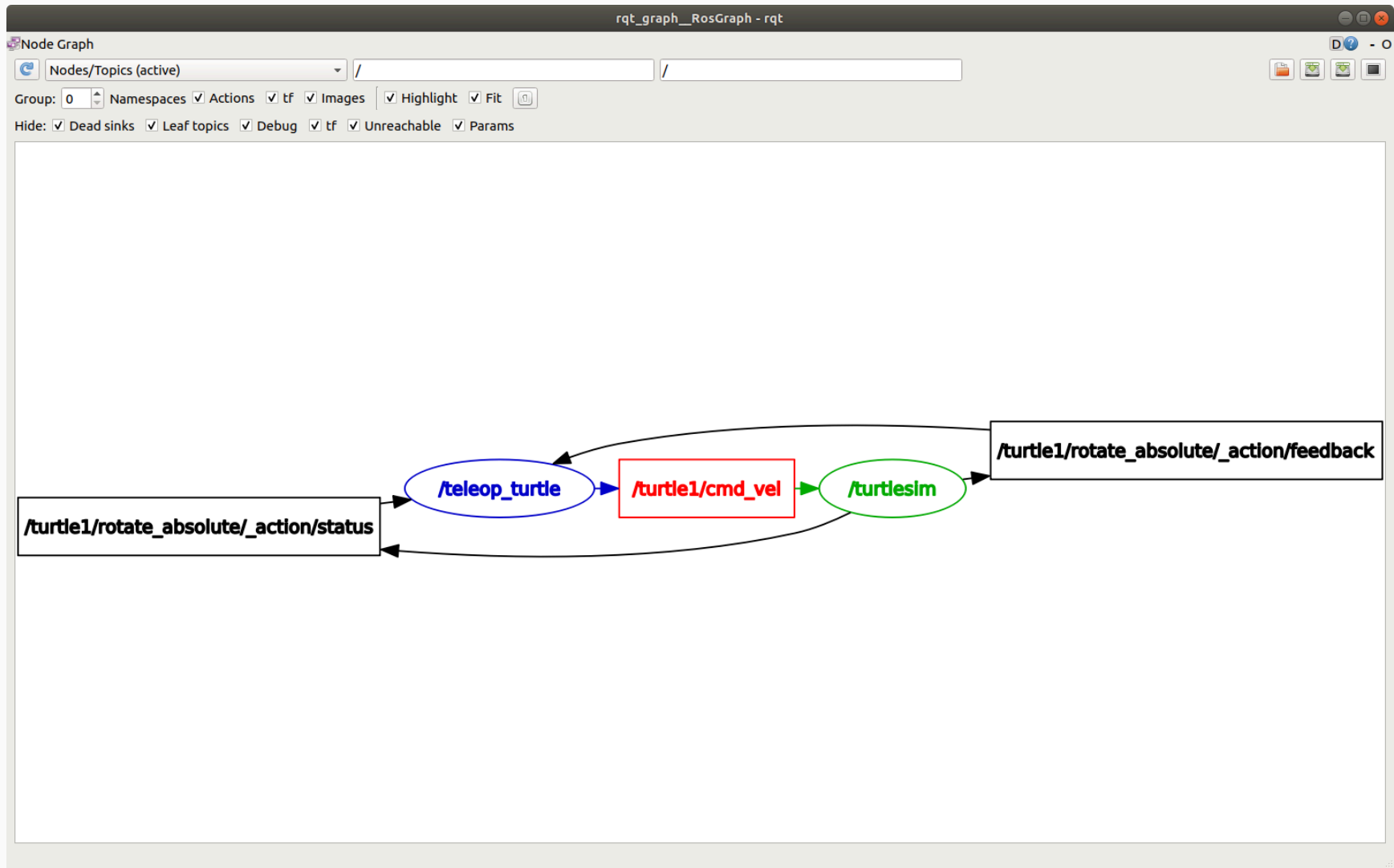
```
# option 1
```

```
rqt_graph
```

```
# option 2
```

```
rqt # select Plugins > Intrsopection > Node Graph
```

- /teleop_turtle node publish keystrokes in /turtle1/cmd_vel
- /turtlesim node is subscribed to /turtle1/cmd_vel



Return list of current topics.

```
# ----- terminal 4 ----- #  
ros2 topic list  
# -----  
/parameter_events  
/turtle1/cmd_vel  
/turtle1/color_sensor  
...
```

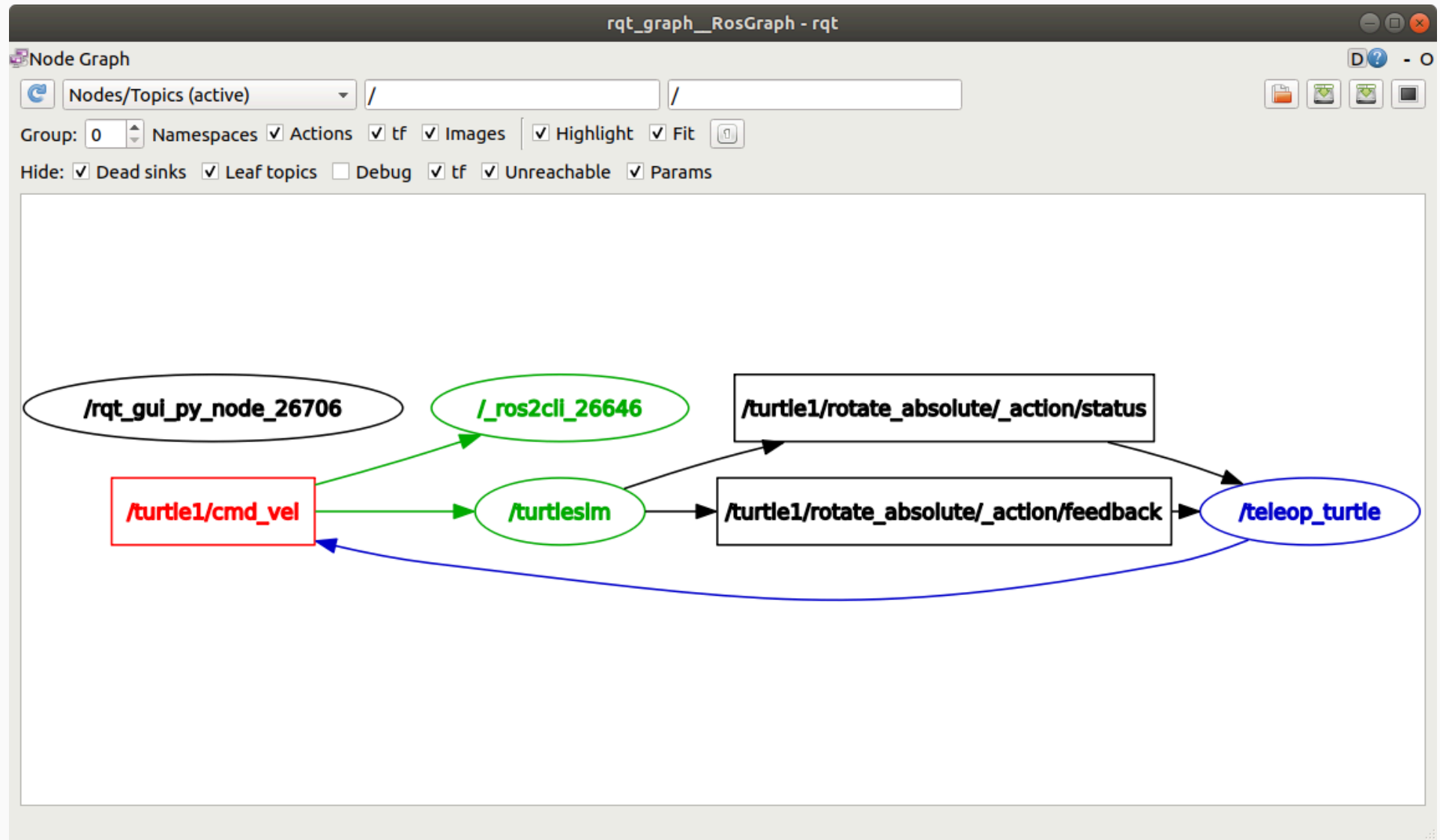
Publishers and subscribers must send and receive the same type of message.

```
# ----- terminal 4 ----- #  
ros2 topic list -t  
# -----  
# `geometry_msgs/msg/Twist` means there is a msg called `Twist`  
# ...  
/turtle1/cmd_vel [geometry_msgs/msg/Twist]  
/turtle1/color_sensor [turtlesim/msg/Color]  
/turtle1/pose [turtlesim/msg/Pose]
```

Shows the data being published on a topic.

```
# ----- terminal 4 ----- #  
ros2 topic echo /turtle1/cmd_vel  
# output (push arrows in terminal with turtle_teleop_key)  
linear:  
  x: 2.0  
  y: 0.0  
  z: 0.0  
angular:  
  x: 0.0  
  y: 0.0  
  z: 0.0  
---
```

A `/_ros2_cli` node is created in `rqt_graph`



Shows the data being published on a topic.

```
# ----- terminal 4 ----- #  
# ros2 topic info <topic_name>  
ros2 topic info /turtle1/cmd_vel  
# -----  
Type: geometry_msgs/msg/Twist  
Publisher count: 1  
Subscription count: 2
```


The interface are the details of the message exchanged.

```
# ----- terminal 4 ----- #
# ros2 interface show <msg_type>
ros2 interface show geometry_msgs/msg/Twist
# -----
# This tells you that the /turtlesim node is expecting a message
# with two vectors, linear and angular, of three elements each.
  Vector3 linear
    float64 x
    float64 y
    float64 z
  Vector3 angular
    float64 x
    float64 y
    float64 z
```

Writes data into the topic

```
# ros2 topic pub <topic_name> <msg_type> <data>
ros2 topic pub --rate 0.5 \
/turtle1/cmd_vel \
geometry_msgs/msg/Twist \
"{linear: {x: 2.0, y: 0.0, z: 0.0}, \
angular: {x: 0.0, y: 0.0, z: 1.8}}"
```

Subcommand hz shows topic rate.

```
# ----- terminal 4 ----- #  
ros2 topic hz /turtle1/pose  
# -----  
average rate: 59.354  
min: 0.005s max: 0.027s std dev: 0.00284s window: 58
```

Subcommand bw shows topic bandwidth.

```
# ----- terminal 4 ----- #  
ros2 topic bw /turtle1/pose  
# -----  
Subscribed to [/turtle1/pose]  
1.51 KB/s from 62 messages  
Message size mean: 0.02 KB min: 0.02 KB max: 0.02 KB
```

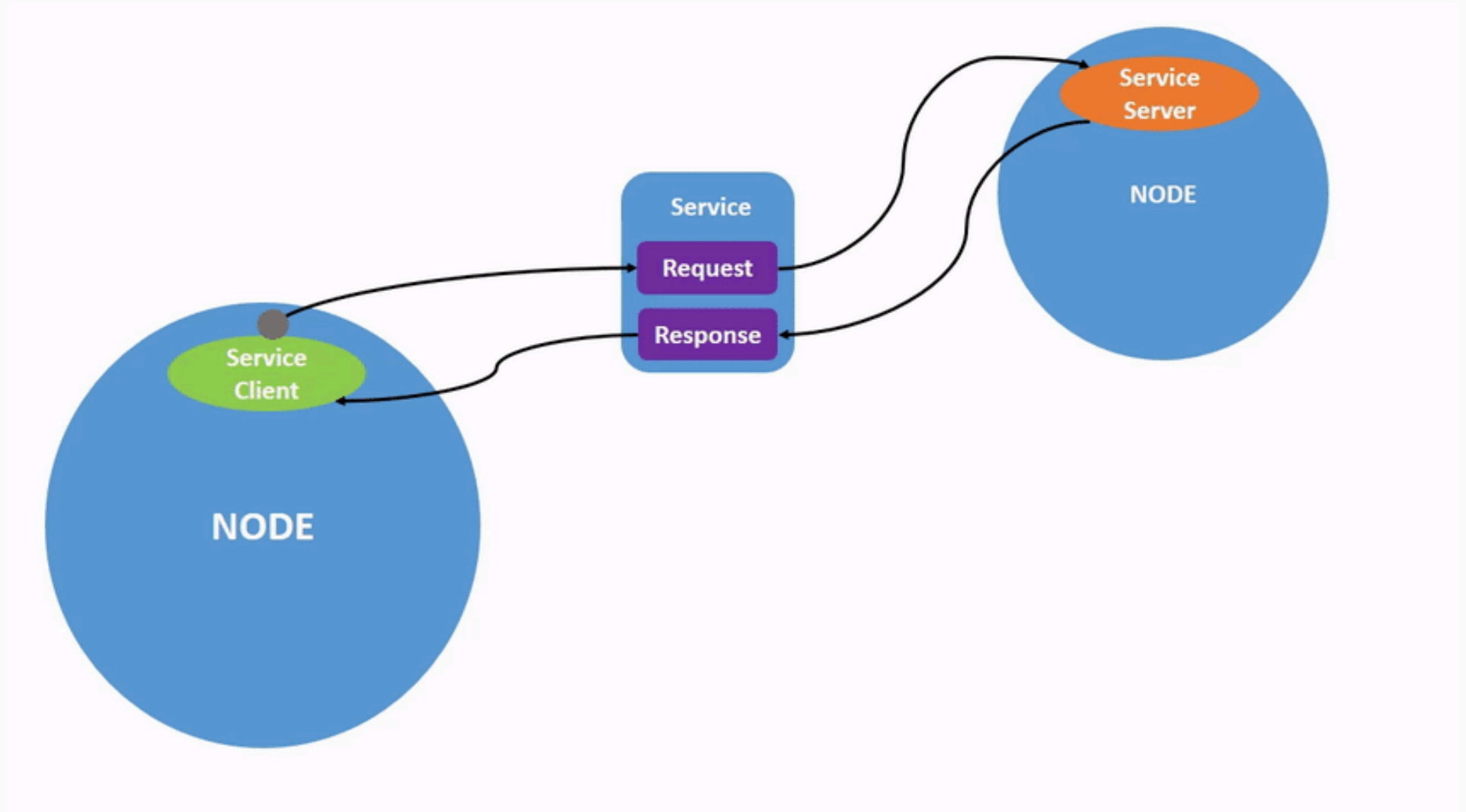
Outputs topics given the message type.

```
# ----- terminal 4 ----- #  
#ros2 topic find <topic_type>  
ros2 topic find geometry_msgs/msg/Twist  
# -----  
/turtle1/cmd_vel
```

Services

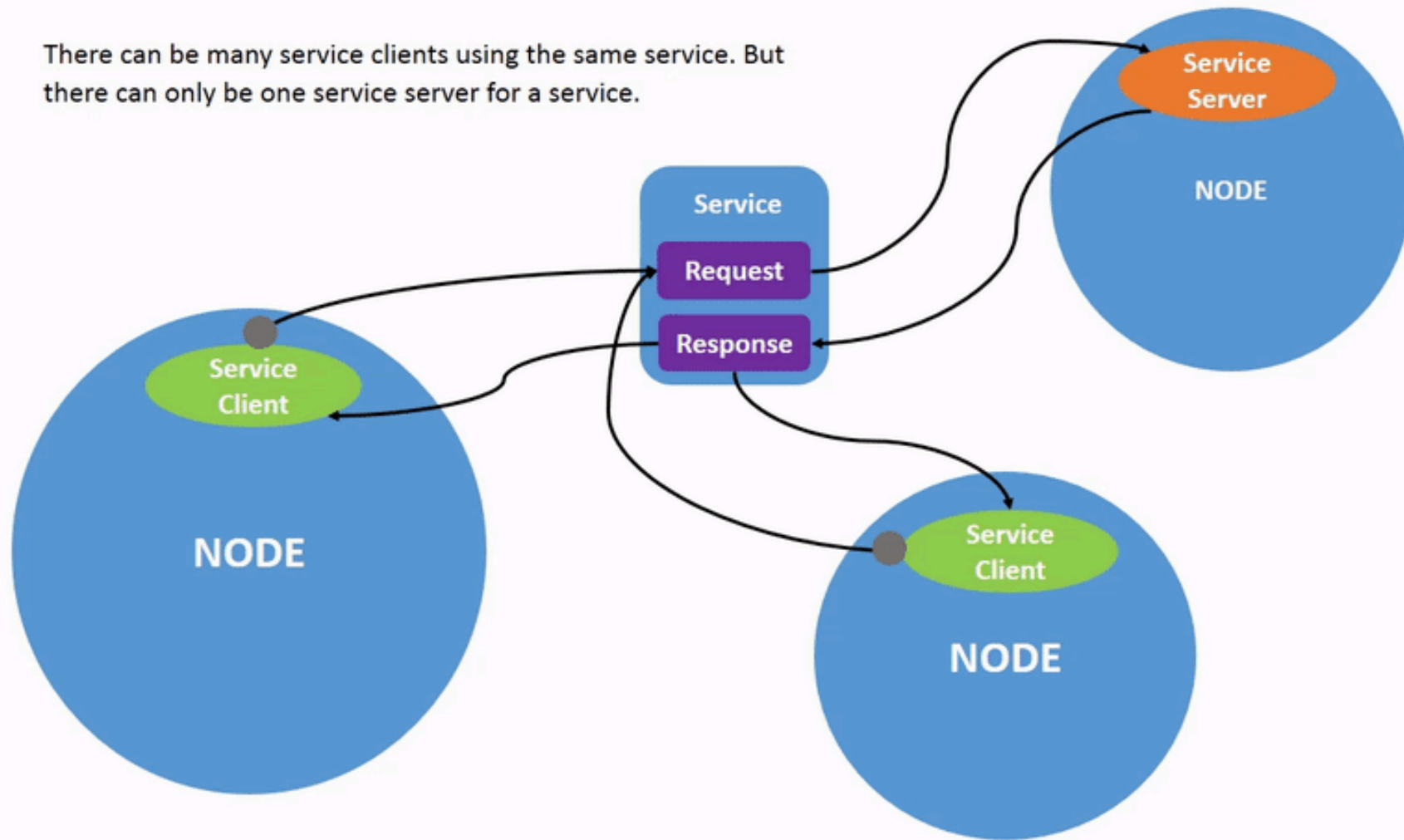


- another method for communication
- based on call-and-response model
- service node response only under client node call
- synchronous



multiple clients

There can be many service clients using the same service. But there can only be one service server for a service.




```
# ----- terminal 1 ----- #  
ros2 run turtlesim turtlesim_node  
  
# ----- terminal 2 ----- #  
ros2 run turtlesim turtle_teleop_key
```

ros2 service list return a list of all the services active

```
# ----- terminal 3 ----- #
```

```
ros2 service list
```

```
# -----
```

```
/clear
```

```
/kill
```

```
/reset
```

```
/spawn
```

```
/teleop_turtle/describe_parameters
```

```
/teleop_turtle/get_parameter_types
```

```
...
```

The type describes the structure of request and response.

```
# ----- terminal 3 ----- #
ros2 service type /clear
# -----
std_srvs/srv/Empty # Empty = sends and receives no data

# ----- terminal 3 ----- #
# `-t` shows the types of all active services.
ros2 service list -t
# -----
/clear [std_srvs/srv/Empty]
/reset [std_srvs/srv/Empty]
/spawn [turtlesim/srv/Spawn]
...
/turtle1/set_pen [turtlesim/srv/SetPen]
/turtle1/teleport_absolute [turtlesim/srv/TeleportAbsolute]
...
```

Find all services of specific type.

```
# ----- terminal 3 ----- #  
ros2 service find std_srvs/srv/Empty  
# -----  
/clear  
/reset
```

Shows structure of the request/response message.

```
# ----- terminal 3 ----- #  
ros2 interface show std_srvs/srv/Empty  
# -----  
# '----' separates the request structure (above)  
# from the response structure (below)  
# In this case, both are empty.  
---
```

```
# ----- terminal 3 ----- #  
ros2 interface show turtlesim/srv/Spawn  
# -----  
float32 x  
float32 y  
float32 theta  
string name  
---  
string name
```

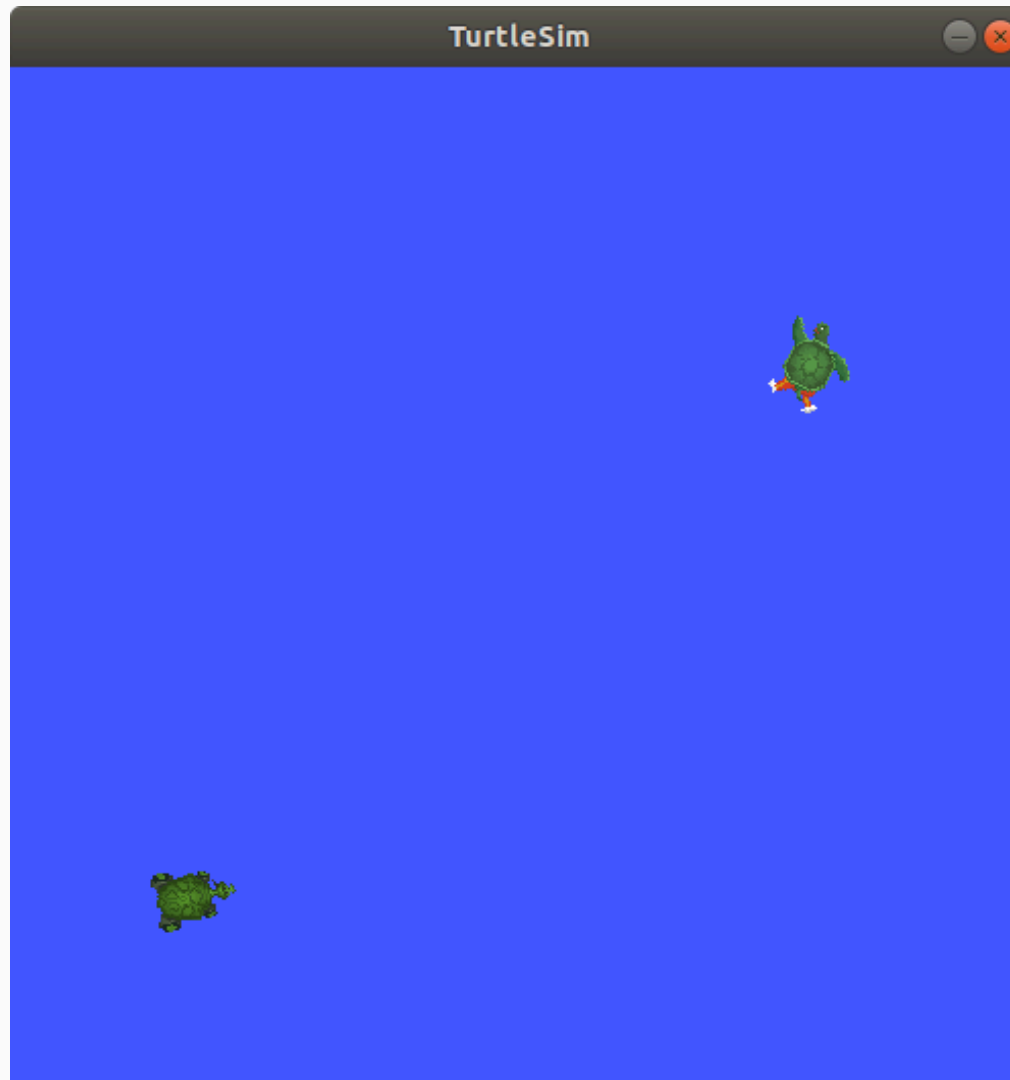
service call 1/

```
# ----- terminal 3 ----- #  
#ros2 service call <service_name> <service_type>  
ros2 service call /clear /std_srvs/srv/Empty
```



service call 2/

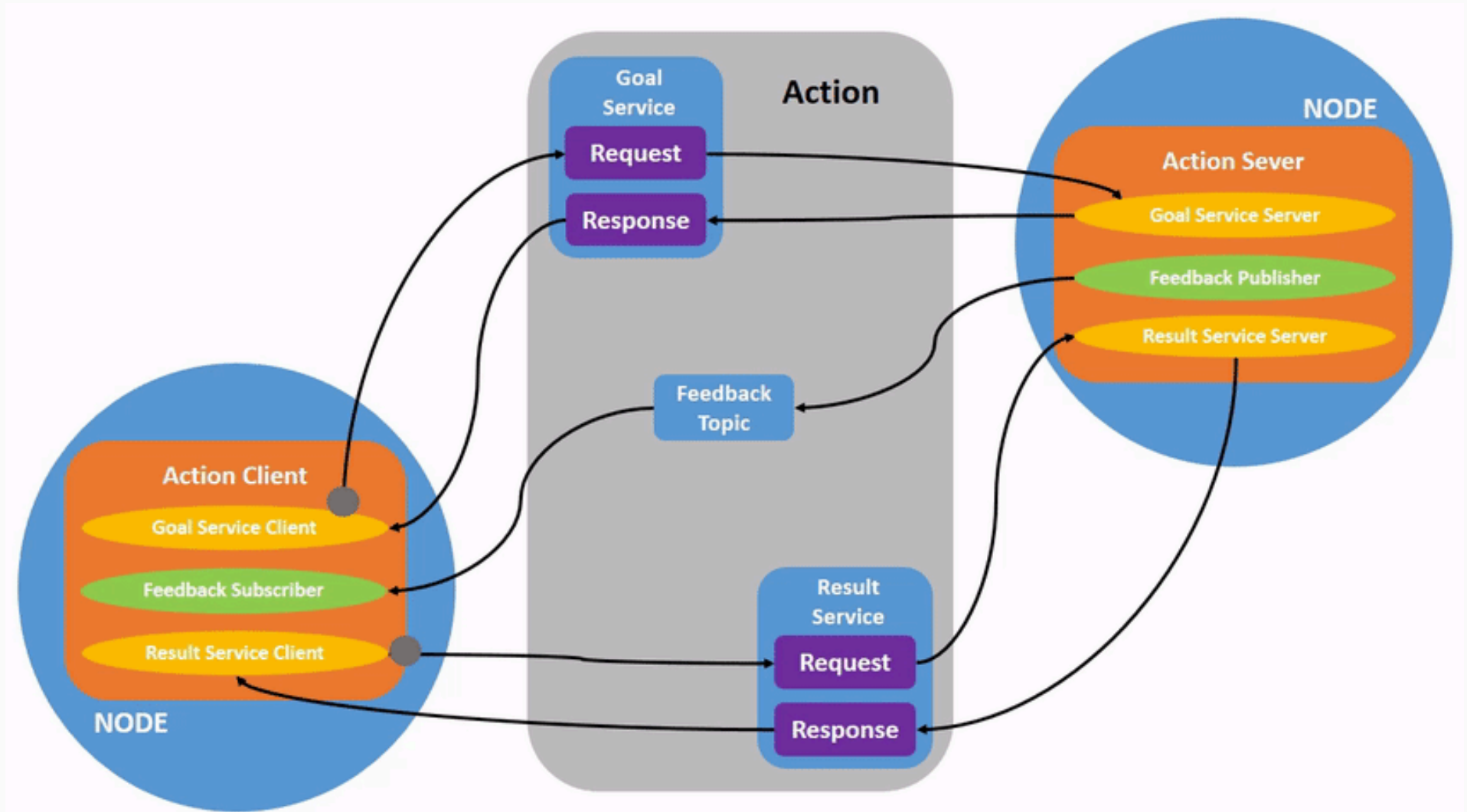
```
# ----- terminal 3 ----- #  
ros2 service call /spawn turtlesim/srv/Spawn \  
"{x: 2, y: 2, theta: 0.2, name: ''}"
```



Actions



- communication for long running tasks
- consists three parts: goal (service), feedback (topic), result (service)
- actions can be canceled at any time



- robot system use actions for navigation
- goal: desired position
- feedback: current position
- result: reached position
- e.g. `/turtle1/rotate_absolute` in turtle sim

```
# ----- terminal 1 ----- #  
ros2 run turtlesim turtlesim_node  
  
# ----- terminal 2 ----- #  
ros2 run turtlesim turtle_teleop_key  
# output  
# Notice that the letter keys G|B|V|C|D|E|R|T  
# form a “box” around the F. F wil cancel execution.  
Use arrow keys to move the turtle.  
Use G|B|V|C|D|E|R|T keys to rotate to absolute orientations.  
'F' to cancel a rotation.
```

- Use G|B|V|C|D|E|R|T
- Each keystroke sends goal to an action server inside /turtlesim node

```
# ----- terminal 2 ----- #
```

```
# Press C, then F key. It will cancel goal mid-execution
```

```
[INFO] [turtlesim]: Rotation goal canceled
```

```
# Press D key, then G key before the rotation can complete
```

```
[WARN] [turtlesim]: Rotation goal received \
```

```
before a previous goal finished. Aborting previous goal
```

List actions provided by a node.

```
# ----- terminal 3 ----- #  
# ros2 node info <node_name>  
ros2 node info /turtlesim
```

```
# output
```

```
/turtlesim
```

```
Subscribers:
```

```
...
```

```
Publishers:
```

```
...
```

```
Service Servers:
```

```
...
```

```
Service Clients:
```

```
...
```

```
Action Servers:
```

```
  /turtle1/rotate_absolute: turtlesim/action/RotateAbsolute
```

```
Action Clients:
```

Identify all the actions in ROS graph

```
# ----- terminal 3 ----- #
```

```
ros2 action list
```

```
# -----
```

```
/turtle1/rotate_absolute
```

```
# ----- terminal 3 ----- #
```

```
ros2 action list -T
```

```
# -----
```

```
/turtle1/rotate_absolute [turtlesim/action/RotateAbsolute]
```

```
# ----- terminal 3 ----- #  
ros2 action info /turtle1/rotate_absolute  
# -----  
# there is one action client (part of /teleop_turtle)  
# there is one action server (part of /turtlesim)  
Action: /turtle1/rotate_absolute  
Action clients: 1  
    /teleop_turtle  
Action servers: 1  
    /turtlesim
```


Shows three parts (goal, feedback, result)

```
# ----- terminal 3 ----- #  
ros2 interface show turtlesim/action/RotateAbsolute  
# output  
# The desired heading in radians  
float32 theta  
---  
# The angular displacement in radians to the starting position  
float32 delta  
---  
# The remaining rotation in radians  
float32 remaining
```

Send action goal from command line.

```
# ----- terminal 3 ----- #  
# ros2 action send_goal <action_name> <action_type> <values>  
ros2 action send_goal \  
/turtle1/rotate_absolute \  
turtlesim/action/RotateAbsolute \  
{theta: 1.57}  
# -----  
Waiting for an action server to become available...  
Sending goal:  
  theta: 1.57  
Goal accepted with ID: f8db8f44410849eaa93d3feb747dd444  
Result:  
  delta: -1.568000316619873  
Goal finished with status: SUCCEEDED
```

- all goals have unique ID
- delta displacement to the starting point
- --feedback shows the feedback of this goal

```
# ----- terminal 3 ----- #
ros2 action send_goal \
/turtle1/rotate_absolute turtlesim/action/RotateAbsolute \
"{theta: -1.57}" --feedback
# -----
Sending goal:
  theta: -1.57
Goal accepted with ID: e6092c831f994afda92f0086f220da27
Feedback:
  remaining: -3.1108222007751465
...
Result:
  delta: 3.1200008392333984
Goal finished with status: SUCCEEDED
```

Glossary of Terms

- node = process
- node perform computation and can exchange data
- data is exchanged by topic or service

- sending data only? use a Topic
- asynchronous data stream
- publisher = node sending information on a topic
- subscriber = node receiving information from topic
- msg = interface

- expecting a response? use a Service
- synchronous request-response mechanism
- client = node requesting
- server = node reponsing
- srv = interface

- use snakecase
- node: indicate the function of the node (e.g. motor_ctrl)
- topic: describe the data emitted by the node (e.g. /motor_ctrl/speed)
- service: describe the action (e.g. /motor_ctrl/shutdown)
- action: goal oriented names (e.g. /motor_ctrl/optimize_consumption)